

Correction des exercices sur les théories du premier ordre, avec Why3

Enseignement à distance de l'Université Marie et Louis Pasteur
Spécification et preuve de programmes

Version du 18 novembre 2025

Ces exercices portent sur les théories du premier ordre et leur implémentation dans Why3.

1 Égalité pour tout type Why3

1. Proposer un code WhyML dont la correction syntaxique justifierait l'affirmation du cours selon laquelle tout type t déclaré en WhyML dispose d'un prédicat binaire d'égalité $=$, sans que l'utilisateur n'ait à le définir.

Le code suivant est suffisant, mais la réponse à la question 2 convient aussi.

```
type t
predicate p (x y : t) = x = y
```

2. Améliorer éventuellement ce code pour que sa preuve avec l'interface en ligne de Why3 démontre automatiquement que ce prédicat binaire d'égalité satisfait les axiomes d'une relation d'équivalence.

Le code suivant formalise comme des buts en WhyML les axiomes de réflexivité, symétrie et transitivité du cours, pour un type t quelconque.

```
type t

goal refl: forall x:t. x = x
goal symm: forall x y:t. x = y → y = x
goal trans: forall x y z:t. x = y → y = z → x = z
```

Ces buts sont démontrés automatiquement par l'interface en ligne de Why3.

3. Après avoir défini un symbole fonctionnel f d'arité 3 pour un type WhyML quelconque t , définir l'axiome de congruence pour ce symbole fonctionnel comme un but ou un lemme WhyML, puis observer si ce but ou lemme est démontré ou pas par l'interface en ligne de Why3.

Le code suivant définit un symbole fonctionnel `f` d'arité 3 pour le type quelconque `t`, et l'axiome de congruence pour ce symbole fonctionnel comme un but WhyML. Ce but est démontré automatiquement par l'interface en ligne de Why3.

```
function f t t t : t

goal congruenceFct: forall x1 x2 x3 y1 y2 y3:t.
  x1 = y1 ∧ x2 = y2 ∧ x3 = y3 → f x1 x2 x3 = f y1 y2 y3
```

4. De manière analogue, définir un code WhyML permettant de vérifier qu'un prédicat d'égalité (=) existe pour les types WhyML `bool` et `list 'a` présentés dans les cours sur les types, et qu'il satisfait les axiomes d'une relation d'équivalence.

Le code suivant traite le cas des booléens

```
goal refl: forall x:bool. x = x
goal symm: forall x y:bool. x = y → y = x
goal trans: forall x y z:bool. x = y → y = z → x = z
```

et le code suivant traite le cas des listes

```
use list.List

goal refl: forall x: list 'a. x = x
goal symm: forall x y: list 'a. x = y → y = x
goal trans: forall x y z: list 'a. x = y → y = z → x = z
```

2 Formules sur les entiers naturels

Dans cet exercice sur les entiers naturels, on considère que la signature fonctionnelle contient une infinité de constantes, qui sont la constante 0 et toutes les séquences finies non vides de chiffres arabes ne commençant pas par 0 : 1, 2, 3, ..., 9, 10, 11, ...

Dans les questions suivantes, on ne parle pas de validité pour toute interprétation des symboles arithmétiques et de l'égalité, mais seulement de validité pour l'interprétation usuelle de l'addition, de la multiplication et de l'égalité dans les entiers naturels.

1. Décrire la signature fonctionnelle et relationnelle d'une théorie dont toutes les expressions représenteraient des entiers naturels, avec une égalité, une addition et une multiplication binaires. Comme il n'y aurait qu'un seul type dans cette théorie, on considère que cette théorie est sans type.

Outre les constantes, la signature fonctionnelle de cette théorie contient les symboles binaires `+` pour l'addition et `×` pour la multiplication. Sa signature relationnelle ne contient que le symbole binaire `=` pour l'égalité. Ces trois symboles sont notés de manière usuelle, infixée.

2. Formaliser dans cette logique l'affirmation " x est pair".

L'affirmation “ x est pair” se traduit par $(\exists y. x = 2 \times y)$.

3. Formaliser par une formule le fait que “tous les entiers ont un suivant” (indication : le suivant de l'entier n peut se noter $n + 1$).

Le fait que “tous les entiers ont un suivant” se traduit par $(\forall n. (\exists p. p = n + 1))$.

4. Formaliser le fait que “0 n'est pas le suivant d'un entier naturel”.

Le fait que “0 n'est pas le suivant d'un entier” se traduit par $\neg(\exists p. 0 = p + 1)$.

5. Formaliser l'affirmation que “0 est le seul entier qui ne soit pas le suivant d'un entier”.

L'affirmation que “0 est le seul entier qui ne soit pas le suivant d'un entier” se traduit par $\neg(\exists p. 0 = p + 1) \wedge (\forall n. (\neg(n = 0) \Rightarrow (\exists p. n = p + 1)))$.

6. Formaliser la proposition que “0 est multiple de chaque nombre entier”.

$\forall x. \exists m. m \times x = 0$.

7. Donner un exemple de formule valide.

Les formules des questions 3, 4, 5 et 6 ci-dessus sont valides. La formule de la question 2 n'est pas valide, parce que, par exemple, l'entier naturel 7 n'est pas pair.

8. Donner un exemple de formule non valide qui n'utilise que l'addition et l'égalité.

$0 + 1 = 0$ est un exemple de formule non valide très simple, n'utilisant que l'addition et l'égalité.

9. Donner un exemple de formule close avec au moins 2 quantificateurs.

La réponse de la question 3 est une formule close avec 2 quantificateurs.

10. Donner un exemple de formule ouverte avec au moins 1 quantificateur.

La réponse de la question 2 est une formule ouverte avec 1 quantificateur. La variable libre est x , la variable liée (par le quantificateur) est y .

11. En WhyML, il n'y a pas de type prédéfini pour les entiers naturels, mais il y a le type `int` pour les entiers relatifs. En utilisant uniquement ce type `int`, ses opérations et son égalité, expliquez comment utiliser Why3 pour vérifier la plupart de vos réponses aux questions précédentes.

Le code suivant permet de vérifier la syntaxe des formules données en réponses aux questions 2 à 6. La preuve des buts justifie la réponse à la question 8, mais seulement une partie de la réponse à la question 7. En effet, les réponses aux questions 4 et 5 sont des formules valides dans les entiers naturels, mais non valides dans les entiers relatifs, où 0 est le successeur de -1 . Le deuxième but regroupe les formules dont la preuve n'est pas automatique dans l'interface en ligne, mais qui sont valides (une démonstration automatique existe, avec des outils plus puissants).

```
use int.Int

predicate q2(x : int) = exists y. x = 2 * y
predicate q3 = forall n. (exists p. p = n+1)
predicate q4 = not (exists p. 0 = p+1)
predicate q5 = not (exists p. 0 = p+1) ∧ (forall n. not (n = 0) →
exists p. n = p+1)
predicate q6 = forall x. exists m. m * x = 0

goal q7 : q3 ∧ not (q2 7)
goal q7moreDifficultToProve : not q4 ∧ not q5 ∧ q6
goal q8: not (0 + 1 = 0)
```

3 Propriétés d'un tableau d'entiers

Soit b un tableau d'entiers Why3 de n éléments. Soit j et k deux entiers entre 0 en $n - 1$: $0 \leq j \leq k < n$. Dans les énoncés suivants, $b[j..k]$ désigne la partie du tableau b entre les indices j et k inclus, mais cette notation n'existe pas dans le langage de Why3.

Définir un prédicat Why3 pour chacune des propriétés suivantes :

1. Tous les éléments de $b[j..k]$ sont nuls.
2. Tous les éléments nuls de $b[0..n - 1]$ sont dans la partie $b[j..k]$.
3. Tous les éléments nuls de $b[0..n - 1]$ ne sont pas dans la partie $b[j..k]$. Une première réponse, évidente, est la négation de la réponse précédente. On demande une autre réponse, qui n'utilise pas la négation.
4. $b[0..n - 1]$ contient au moins deux zéros.
5. $b[0..n - 1]$ contient exactement deux zéros.
6. $b[0..n - 1]$ contient au plus deux zéros.

Toutes les réponses sont dans le listing 1.

```
use int.Int
use array.Array

predicate q1 (b:array int) (j k:int) = forall i:int. j ≤ i ≤ k → b[i] = 0

predicate q2 (b:array int) (j k:int) =
  forall i:int. 0 ≤ i < b.length → b[i] = 0 → j ≤ i ≤ k
```

```

predicate q3 (b:array int) (j k:int) =
  exists i:int. (0 ≤ i < j ∨ k < i < b.length) ∧ b[i] = 0

predicate q4 (b:array int) (j k:int) =
  exists i:int. 0 ≤ i < b.length ∧ b[i] = 0 ∧
    (exists j:int. 0 ≤ j < b.length ∧ b[j] = 0 ∧ not (j = i))

predicate q5 (b:array int) (j k:int) =
  exists i:int. 0 ≤ i ≤ b.length-1 ∧ b[i] = 0 ∧
    (exists j:int. 0 ≤ j ≤ b.length-1 ∧ b[j] = 0 ∧ not (j = i) ∧
      (forall m:int. 0 ≤ m ≤ b.length-1 ∧ not (m = i) ∧ not (m = j)
        → not (b[m] = 0)))

predicate q6 (b:array int) (j k:int) =
  forall i:int. 0 ≤ i ≤ b.length-1 → b[i] = 0 →
    (forall j:int. 0 ≤ j ≤ b.length-1 → not (i = j) → b[j] = 0 →
      (forall k:int. 0 ≤ k ≤ b.length-1 → not (k = i) → not (k = j) →
        not (b[k] = 0)))

```

Listing 1 – Propriétés de tableaux d'entiers.